RT-IoT2022 Multi-Class IoT Attack Detection

Project Report — CSE 445 (Machine Learning), Section 8

Faculty: Dr. Jilan Samiuddin | TA: Bijoy Deb Babu

[Your Name — Student ID | Group Members (if applicable)]

1. Problem Statement & Dataset Description

IoT networks are increasingly targeted by cyberattacks such as brute-force attempts, denial-of-service floods, and reconnaissance scans. This project builds a supervised machine learning pipeline that classifies IoT network traffic into one of 12 categories (normal device traffic plus 11 attack/traffic types) based on flow-level statistical features, so that malicious traffic can be automatically distinguished from benign traffic.

1.1 Dataset Overview

Property	Description
Dataset name	RT-IoT2022
Source	UCI Machine Learning Repository (dataset id 942)
Type	Tabular, multivariate, real-time IoT network flow data
Instances	123,117
Features	84 (flow-level statistics: durations, packet counts, inter-arrival times, protocol/service info, etc.)
Target classes	12 (normal IoT traffic + 11 attack/traffic categories, e.g. brute-force SSH, DDoS via Hping/Slowloris, Nmap scans)
Task	Multi-class classification

The dataset was captured from a real-time IoT testbed using the Zeek network monitoring tool and the Flowmeter plugin, combining traffic from IoT devices (ThingSpeak-LED, Wipro-Bulb, MQTT-Temp) with simulated attacks.

2. EDA, Cleaning & Preprocessing Explanation

2.1 Data Cleaning

- Missing values: checked per column; any found were imputed (median for numeric, mode for categorical) rather than dropped, to avoid losing rare-attack rows.
- Duplicate rows: exact duplicates were removed to prevent identical rows from leaking across the train/validation/test split.
- Infinite/invalid values (occasionally present in flow-rate features) were replaced with the column's finite maximum.

[Insert actual counts from your run: total missing values = ___, duplicate rows removed = ___, final cleaned shape = ___ rows x ___ columns]

2.2 Exploratory Data Analysis — Key Findings

- Class distribution: the dataset is imbalanced — the largest class has substantially more samples than the smallest.

[Insert your class-distribution bar chart here + the largest/smallest class names and the imbalance ratio you computed]

- Feature distributions: at least three numeric flow features were plotted; most show a strong right-skew typical of network traffic (many short/normal flows, a long tail of unusual ones).

[Insert your 3 feature-distribution histograms here]

- Correlation heatmap: a subset of numerical features was checked for multicollinearity.

[Insert your correlation heatmap here and note any highly correlated feature pairs]

- Outliers: IQR-based checks found a non-trivial share of outliers in flow-timing features; these were kept rather than removed, since they likely correspond to genuine attack behaviour.

2.3 Preprocessing & Data Split

- Target labels were label-encoded; remaining categorical feature columns (protocol/service) were one-hot encoded.
- A stratified 70% / 15% / 15% train / validation / test split was used to preserve class ratios across splits.
- StandardScaler was fit only on the training set, then applied to validation and test sets — preventing data leakage.

3. Modeling Methodology

Three classification models covered in the course were trained on the identical data split:

- Logistic Regression (class_weight='balanced') — linear baseline.
- Random Forest (200 trees, class_weight='balanced_subsample') — bagged non-linear ensemble, generally robust to noisy/imbalanced tabular data.
- MLP Classifier (2 hidden layers, 128 and 64 units) — trained epoch-by-epoch for 30 epochs, with training and validation metrics (loss, accuracy, precision, recall, macro-F1) recorded every epoch to monitor convergence and overfitting.

The validation set was used only to monitor the MLP's training curves and to sanity-check model behaviour before final testing; the test set was used exclusively for the final, one-time evaluation reported below.

4. Results, Graphs & Evaluation

4.1 Metric Comparison Table

[Paste your results_df table here (Accuracy, Macro Precision/Recall/F1, Weighted Precision/Recall/F1 for all 3 models)]

4.2 Comparison Graph

[Insert the bar chart comparing Accuracy / Macro Precision / Macro Recall / Macro F1 / Weighted F1 across the 3 models]

4.3 Confusion Matrices

[Insert the 3 confusion-matrix heatmaps (one per model) and describe the most frequently confused class pairs]

4.4 Epoch-wise Curves (MLP)

[Insert the training-vs-validation loss/accuracy/precision/recall/F1 curves for the MLP and comment on whether it over/under-fits]

4.5 Classification Reports

[Paste the per-class precision/recall/F1 classification report for each model (or at least the selected best model)]

5. Discussion, Best Model & Limitations

5.1 Best Model Selection

The best model was selected using balanced reasoning across all major metrics — specifically, the average rank across Macro Precision, Macro Recall, Macro F1, and Weighted F1 — rather than relying on accuracy alone, since

accuracy can look misleadingly high on an imbalanced 12-class dataset while minority attack classes are still poorly detected.

[State which model was selected as best and its metric values, referencing the average-rank table from the notebook]

5.2 Class-wise Performance & Common Misclassifications

[Identify which 2-3 classes are hardest to classify / most often confused with each other, based on the confusion matrices]

5.3 Model Limitations

- Logistic Regression: assumes linear separability; likely underperforms on classes whose flow-feature distributions overlap non-linearly.
- Random Forest: strong general performance but can overfit very small minority classes and is less interpretable at scale.
- MLP: sensitive to hyperparameters (layer sizes, learning rate, epoch count) and scaling; may need more epochs/data to fully converge on rare classes.
- General: the dataset comes from one specific IoT testbed and attack simulation setup, so performance may not generalize to other devices, network conditions, or unseen ('zero-day') attack types.

5.4 Possible Improvements

- Class-balancing techniques (SMOTE, weighted resampling) to improve minority-class recall.
- Systematic hyperparameter tuning (grid/random search) on the validation set.
- Ensembling the three models, or trying gradient-boosted trees (e.g. XGBoost/LightGBM), for a potential further boost.

References

B. S. Sharmila and R. Nagapadma (2023). RT-IoT2022 [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5P338

[Add any additional external resources, articles, or library documentation you consulted, with citations, per the academic integrity requirements]